CS-471: Machine Learning

Week 7 — C4.5 Missing Values, CART Regression, Decision Tree Pruning, Tree Ensembles, Bagging, Random Forests, Boosting & XGBoost

Instructor: Dr. Hashir Kiani Sources: Lectures 7-1 & 7-2

Comprehensive Study Notes

Week 7

Contents

1 How C4.5 Handles Missing Values **2**
- 1.1 During Split Selection . . . 2
- 1.2 During Tree Construction . . . 2
- 1.3 During Prediction . . . 2

2 CART for Regression — Full Worked Example **3**
- 2.1 Recap: The CART Regression Objective . . . 3
- 2.2 Dataset . . . 4
- 2.3 Step 1: Compute Overall MSE . . . 4
- 2.4 Step 2: Evaluate Candidate Splits on Study Hours . . . 4
- 2.5 Optimal Split and Predictions . . . 5

3 Decision Tree Pruning **5**
- 3.1 What Is Pruning and Why Do We Need It? . . . 5
- 3.2 Two Types of Pruning . . . 5
  - 3.2.1 Pre-Pruning (Early Stopping) . . . 5
  - 3.2.2 Post-Pruning (Error-Based Pruning) . . . 6

4 Chi-Squared (χ^2) Pruning — Pre-Pruning Test **6**
- 4.1 The Core Idea . . . 6
- 4.2 Computing the χ^2 Statistic . . . 6
- 4.3 Degrees of Freedom and Critical Value . . . 7
- 4.4 Chi-Squared Critical Value Table . . . 7

5 C4.5 Post-Pruning — Error-Based Method **8**
- 5.1 The Four Steps of C4.5 Post-Pruning . . . 8
  - 5.1.1 Step 1: Grow the Full Decision Tree . . . 8
  - 5.1.2 Step 2: Compute Observed Error for Each Node . . . 8
  - 5.1.3 Step 3: Compute Estimated Error with Confidence Interval Adjustment . . . 8
  - 5.1.4 Step 4: Prune if Leaf Error $\leq$ Subtree Error . . . 8

5.2 Worked Example of C4.5 Pruning 9

6 Decision Tree Sensitivity and the Need for Ensembles 10
6.1 The Core Problem: High Variance 10
6.2 Solution: Tree Ensembles 10

7 Bagging and Random Forests 10
7.1 Bagging (Bootstrap AGGregatING) 10
7.2 Random Forest 11

8 Boosting and XGBoost 12
8.1 The Boosting Concept 12
8.2 XGBoost (Extreme Gradient Boosting) 13
8.3 XGBoost Algorithm — Step by Step 13
8.4 XGBoost Worked Example 14

9 Comparison: Bagging vs. Random Forest vs. Boosting 15

10 Practice Questions 15

11 Key Takeaways — Week 7 17

1 How C4.5 Handles Missing Values

Real-world datasets frequently contain missing feature values. Unlike ID3 (which cannot handle them at all), C4.5 uses a **probabilistic fraction-based strategy** so that no example is simply discarded.

1.1 During Split Selection

The Information Gain formula is adjusted to account for missing data:

$$\boxed{\text{Gain}(A) = \frac{N_{\text{known}}}{N} \times \text{Gain}_{\text{on known subset}}}$$

- N = total number of examples at the current node.
- N_{known} = number of examples that have a value for attribute A.
- The fraction N_{known}/N acts as a *confidence penalty*: if most values are missing, the effective gain is reduced.

Intuition: If only 3 out of 12 examples have a value for attribute A, then even a perfect split on those 3 gives only $(3/12) = 0.25$ times its face value. The attribute is treated as less reliable.

1.2 During Tree Construction

When building the tree, an example with a missing value for the chosen split attribute is **distributed fractionally across all child branches**:

1. Compute the fraction of *known* examples going to each branch (e.g., 4 out of 9 known examples go to "Rainy").
2. Send the missing-value example to *each* branch with a weight proportional to those fractions.
3. Fractional examples contribute proportionally to class counts, entropy calculations, and predictions.

1.3 During Prediction

When a new test example has a missing value for an attribute at some node:

1. Send the example down **all branches simultaneously.**
2. Each branch returns a probability distribution over classes.
3. The branch probabilities are **weighted** by the fraction of training examples that went down each branch.
4. The final prediction is the class with the **highest weighted probability**.

Key Idea

Why this is clever: Instead of guessing a value or ignoring the example, C4.5 says "I will partially believe the outcome of every branch proportionally to how likely each value of the attribute is." This is a principled probabilistic strategy rather than simple imputation.

Example

Student Pass/Fail with missing Attendance data:

Student	Study Hours	Attendance (%)	Label
A	1	50	Fail
B	5	80	Pass
C	6	85	Pass
D	2	40	Fail
E	3	60	Fail
F	8	90	Pass

All 6 examples have values, so it is a clean dataset. Now suppose Student G has Study Hours = 4 but Attendance is missing.
During prediction: Training data shows 50% of examples have Study Hours ≤ 3.5 (A, D, E) and 50% have Study Hours > 3.5 (B, C, F).
G is sent to *both* branches:

- Branch ≤ 3.5 (weight 0.5): majority class = Fail.
- Branch > 3.5 (weight 0.5): majority class = Pass.

Weighted probability: $P(\text{Pass}) = 0.5 \times 0 + 0.5 \times 1 = 0.5$; $P(\text{Fail}) = 0.5$. Tie $\Rightarrow$ use prior (majority class in training set is Fail with 3 vs 3, so still a tie — predict either, or use further tie-breaking).

2 CART for Regression — Full Worked Example

2.1 Recap: The CART Regression Objective

For a regression task (continuous output y), CART splits the data to minimise the **weighted Mean Squared Error (MSE)** at each node. At each leaf, the prediction is the **mean** of the target values.

$$\text{MSE}(S) = \frac{1}{|S|}\sum_{i\in S}(y_i - \bar{y})^2, \quad \bar{y} = \frac{1}{|S|}\sum_{i\in S} y_i$$

The best split is $\arg\min_{A,t}\big[\text{weighted MSE after split on } (A \leq t)\big]$.

2.2 Dataset

Student	Study Hours (x_1)	Attendance (%) (x_2)	Marks (y)
A	1	50	35
B	5	80	75
C	6	85	82
D	2	40	40
E	3	60	55
F	8	90	92

2.3 Step 1: Compute Overall MSE

$\bar{y} = (35 + 75 + 82 + 40 + 55 + 92)/6 = 379/6 \approx 63.2$

$$\text{MSE}_{\text{all}} = \frac{1}{6}\left[(35-63.2)^2+(75-63.2)^2+(82-63.2)^2+(40-63.2)^2+(55-63.2)^2+(92-63.2)^2\right]$$

$$= \frac{795.24 + 139.24 + 353.44 + 538.24 + 67.24 + 829.44}{6} = \frac{2723}{6} \approx \mathbf{453.8}$$

2.4 Step 2: Evaluate Candidate Splits on Study Hours

Sorted Study Hours: 1, 2, 3, 5, 6, 8. Candidate thresholds: 1.5, 2.5, 3.5, 4.0, 5.5, 7.0.

Threshold = 3.5 (Split A, D, E vs B, C, F):

- Left (≤ 3.5): A(35), D(40), E(55). $\bar{y}_L = 130/3 \approx 43.3$. $\text{MSE}_L = [(35 - 43.3)^2 + (40 - 43.3)^2 + (55 - 43.3)^2]/3 = [68.9 + 10.9 + 136.9]/3 = 216.7/3 \approx 72.2$
- Right (> 3.5): B(75), C(82), F(92). $\bar{y}_R = 249/3 = 83$. $\text{MSE}_R = [(75 - 83)^2 + (82 - 83)^2 + (92 - 83)^2]/3 = [64 + 1 + 81]/3 = 146/3 \approx 48.7$

$$\text{Weighted MSE}_{3.5} = \frac{3}{6}(72.2) + \frac{3}{6}(48.7) = 36.1 + 24.3 = \mathbf{60.4}$$

Reduction: $453.8 - 60.4 = 393.4$ — an enormous improvement.

Threshold = 1.5 (Split A vs D,E,B,C,F):

- Left (≤ 1.5): A(35). $\text{MSE}_L = 0$.
- Right (> 1.5): D(40), E(55), B(75), C(82), F(92). $\bar{y}_R = 344/5 = 68.8$. $\text{MSE}_R \approx 326.6$

Weighted $\text{MSE}_{1.5} = (1/6)(0) + (5/6)(326.6) \approx 272.2$

Threshold 3.5 is clearly superior.

2.5 Optimal Split and Predictions

Condition	Examples	Predicted Marks
Study Hours ≤ 3.5	A, D, E	$\hat{y} = 43.3$
Study Hours > 3.5	B, C, F	$\hat{y} = 83.0$

Key Idea

The CART regression tree predicts the *mean* label of examples in each leaf. A student studying ≤ 3.5 hours is predicted to score 43 marks; one studying more is predicted to score 83 marks. Deeper splits would further refine these predictions.

3 Decision Tree Pruning

3.1 What Is Pruning and Why Do We Need It?

An **unpruned** decision tree grows until every leaf is pure (or all attributes are exhausted). On noisy real-world data, this causes **overfitting**: the tree memorises training noise and generalises poorly.

Pruning removes branches of the tree that are *statistically unreliable* or provide little real improvement over simply predicting the majority class.

- **Benefit 1:** Reduces overfitting and improves generalisation to unseen data.
- **Benefit 2:** Produces a smaller, simpler tree that is easier to interpret and faster to evaluate.
- **Key insight:** A complex subtree that "accidentally" separates two noisy training examples is not capturing a real pattern — it should be collapsed to a leaf.

3.2 Two Types of Pruning

3.2.1 Pre-Pruning (Early Stopping)

Stop growing the tree *before* it becomes too complex:

- Set a **maximum depth**: never grow beyond d levels.
- Set **minimum samples per node**: do not split a node with fewer than k examples.
- Set a **minimum impurity reduction threshold**: only split if $\Delta\text{Gini} > \delta$.
- Apply a **statistical significance test** (e.g. χ^2 test): only split if the attribute is statistically relevant.

Advantage: Fast; avoids building large trees.
Disadvantage: May stop too early (misses patterns that require multiple splits to emerge).

3.2.2 Post-Pruning (Error-Based Pruning)

Grow the full tree first; then *remove* branches that do not help:

- Build the complete tree.
- Starting from the leaves, evaluate each internal node.
- If replacing a subtree with a single leaf node *does not significantly increase* the estimated error, prune it.
- Repeat bottom-up until no improvement is possible.

C4.5 uses error-based post-pruning.
Advantage: More accurate than pre-pruning (sees the full tree before deciding).
Disadvantage: More computationally expensive.

4 Chi-Squared (χ^2) Pruning — Pre-Pruning Test

4.1 The Core Idea

A node that tests an **irrelevant attribute** will still show some apparent class separation due to random chance. The χ^2 test asks: "Is the class separation we see at this node *significantly* better than what we would expect by chance, *if the attribute were irrelevant*?"

- **Null hypothesis (H_0):** The attribute being tested is irrelevant (class distribution is the same in all branches).
- If we *reject* H_0 (significant result): keep the split — the attribute is informative.
- If we *fail to reject* H_0 (insignificant result): prune the split — the attribute adds no real information.

4.2 Computing the χ^2 Statistic

Suppose a node splits into d subsets. For subset k, let:

- p_k = actual number of positive (class 1) examples.
- n_k = actual number of negative (class 0) examples.
- p, n = total positives and negatives across the whole node.

The **expected** counts *if the attribute were irrelevant* — i.e., if class distribution in each branch mirrors the overall distribution:

$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n}, \qquad \hat{n}_k = n \times \frac{p_k + n_k}{p + n}$$

The χ^2 statistic summed over all d subsets:

$$\boxed{\chi^2 = \sum_{k=1}^{d} \left[\frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k} \right]}$$

Key Idea

Intuition: Each term $(x - \hat{x})^2/\hat{x}$ measures how much the *actual* count differs from the *expected* count. If the attribute is truly irrelevant, actual $\approx$ expected and $\chi^2 \approx 0$. A large χ^2 means the actual split differs greatly from what randomness alone would produce — strong evidence the attribute is relevant.

4.3 Degrees of Freedom and Critical Value

- Degrees of freedom $= d - 1$, where d is the number of subsets (branches).
- Choose a significance level α (common choices: 0.05 or 0.10).
- Look up the critical value χ^2_{crit} from the χ^2 table.

Decision rule:

- If $\chi^2 > \chi^2_{\text{crit}}$: **Keep** the split (attribute is relevant).
- If $\chi^2 \leq \chi^2_{\text{crit}}$: **Prune** (attribute is irrelevant at this significance level).

4.4 Chi-Squared Critical Value Table

Degrees of	Significance Level α				
Freedom	0.10	0.05	0.02	0.01	0.001
1	2.706	3.841	5.412	6.635	10.827
2	4.605	5.991	7.824	9.210	13.815
3	6.251	7.815	9.837	11.345	16.268
4	7.779	9.488	11.668	13.277	18.465
5	9.236	11.070	13.388	15.086	20.517

Example

Quick χ^2 calculation: Suppose a binary split yields:

- Branch 1: $p_1 = 5$, $n_1 = 1$ (6 total)
- Branch 2: $p_2 = 2$, $n_2 = 4$ (6 total)
- Total: $p = 7$, $n = 5$, $N = 12$.

Expected values:

$$\hat{p}_1 = 7 \times \frac{6}{12} = 3.5, \quad \hat{n}_1 = 5 \times \frac{6}{12} = 2.5$$

$$\hat{p}_2 = 7 \times \frac{6}{12} = 3.5, \quad \hat{n}_2 = 5 \times \frac{6}{12} = 2.5$$

$$\chi^2 = \frac{(5-3.5)^2}{3.5} + \frac{(1-2.5)^2}{2.5} + \frac{(2-3.5)^2}{3.5} + \frac{(4-2.5)^2}{2.5} = \frac{2.25}{3.5} + \frac{2.25}{2.5} + \frac{2.25}{3.5} + \frac{2.25}{2.5}$$

$$= 0.643 + 0.900 + 0.643 + 0.900 = \mathbf{3.086}$$

Degrees of freedom $= 2 - 1 = 1$. Critical value at $\alpha = 0.05$ with DF $= 1$: $\chi^2_{\text{crit}} = 3.841$.
Since $3.086 < 3.841$: **Fail to reject** H_0 — attribute appears statistically irrelevant; prune this split.

5 C4.5 Post-Pruning — Error-Based Method

5.1 The Four Steps of C4.5 Post-Pruning

5.1.1 Step 1: Grow the Full Decision Tree

Expand the tree to full depth using Gain Ratio splitting. No pruning yet.

5.1.2 Step 2: Compute Observed Error for Each Node

For any internal node or leaf, the *observed training error* is:

$$\boxed{\text{ObservedError} = \frac{E}{N}}$$

where E = number of misclassified training examples at that node, and N = total training examples at that node.

The prediction at the node is the *majority class*, so E = (total examples) − (majority class count).

5.1.3 Step 3: Compute Estimated Error with Confidence Interval Adjustment

Training error tends to be *optimistic* (too low). C4.5 adds a confidence adjustment to get a more conservative, *upper-bound* estimate:

$$\boxed{\text{EstimatedError} = \text{ObservedError} + \text{ConfidenceAdjustment}}$$

The confidence adjustment uses the upper bound of a binomial proportion confidence interval at a chosen level (e.g., 25%):

$$\text{ConfAdj} = z_{\alpha/2}\sqrt{\frac{\text{ObservedError}(1 - \text{ObservedError})}{N}}$$

where $z_{\alpha/2}$ is the normal quantile (for 25% confidence, $z \approx 0.69$).

5.1.4 Step 4: Prune if Leaf Error ≤ Subtree Error

Compare the estimated error of replacing the subtree with a leaf versus keeping the subtree:

- If Error(leaf) ≤ Error(subtree): Replace the subtree with a leaf. **(Prune it.)**

- If Error(leaf) > Error(subtree): Keep the subtree. **(Do not prune.)**

Repeat this bottom-up comparison until no further pruning is beneficial.

Key Idea

Why add a confidence adjustment? On training data, even a random split can reduce error. The confidence interval adjustment "pessimises" the estimate, accounting for the fact that smaller nodes have less reliable error estimates (high variance). A leaf with 2 examples where 1 is misclassified looks like 50% error; the CI makes the estimate much higher to reflect that 2 examples is insufficient evidence.

5.2 Worked Example of C4.5 Pruning

Tree structure (extracted from lecture):

The full tree splits first on **Study Hours per Week**:

- Branch > 15: leads to a leaf node with 1 Pass, 1 Fail.
- Branch $\leq$ 15: leads to an internal node splitting on **Extra-Curricular Participation**, with three sub-leaves:

Sub-leaf (Extra-Curr.)	Pass	Fail	Predicted	Training Error
None	5	2	Pass	$2/7 \approx 0.286$
High	2	4	Fail	$2/6 \approx 0.333$
Moderate	2	1	Pass	$1/3 \approx 0.333$

Total examples at the Extra-Curricular node: $7 + 6 + 3 = 16$ with 9 Pass, 7 Fail.

Subtree training error (sum of leaf errors): $2 + 2 + 1 = 5$ misclassifications out of 16. Error(subtree) $= 5/16 = 0.3125$.

Leaf error (if we replace with a single leaf predicting the majority class: Pass, since $9 > 7$): 7 misclassifications out of 16. Error(leaf) $= 7/16 = 0.4375$.

Comparison: Error(leaf) > Error(subtree), i.e., $0.4375 > 0.3125$: **Do not prune**. The Extra-Curricular Participation subtree provides genuinely useful splits compared to the majority-class leaf.

Common Mistake / Pitfall

In C4.5 the comparison uses *estimated* error (with confidence adjustment), not just observed training error. The confidence adjustment can flip the pruning decision when subsets are very small — a node with 3 examples and 1 error (33% observed) may have a very wide CI, making its estimated error approach 50% or more.

6 Decision Tree Sensitivity and the Need for Ensembles

6.1 The Core Problem: High Variance

Decision trees are **high-variance** models: a small change in training data — such as adding, removing, or mislabelling just one or two examples — can produce an *entirely different* tree structure.

- A different root split ⇒ completely different branches and predictions.
- This instability leads to **high variance** in predictions across different training sets drawn from the same population.
- Single deep trees often overfit.

Key Idea

Why high variance? The greedy top-down algorithm makes a single permanent decision at each node. A slightly different dataset may favour a different attribute at the root, and this propagates as a cascade of different decisions through the entire tree.

6.2 Solution: Tree Ensembles

Ensemble methods combine many decision trees so that individual errors cancel out.

- Each individual tree is a "weak learner" — noisy but better than random.
- Combining many trees averages out the noise, producing a "strong learner" with **lower variance**.
- The key requirement: the trees must be trained on *different* but representative subsets of the data so their errors are not correlated.

How predictions are combined:

- **Classification:** Majority voting — choose the class that most trees predict.
- **Regression:** Averaging — take the mean of all trees' numerical predictions.

7 Bagging and Random Forests

7.1 Bagging (Bootstrap AGGregatING)

Bagging creates diversity among trees by training each one on a different random subset of the training data.

Algorithm:

1. Let the training set have m examples.
2. Create B **bootstrap samples**: each is a random sample of m examples drawn **with replacement** from the original training set. (Some examples appear multiple times; some are absent.)

3. Train an independent decision tree on each bootstrap sample.
4. Combine predictions:
   - Classification: majority vote.
   - Regression: average.

Key Idea

Bootstrap Sampling — what it means: Sampling *with replacement* means each draw is independent. In a sample of m examples from a dataset of size m, on average about 63.2% of original examples appear at least once (the rest, $\approx 36.8\%$, are absent — these are called *out-of-bag* examples and can be used for free validation).

Why does bagging reduce variance?
Each tree is trained on a slightly different dataset and makes slightly different errors. Majority voting averages out these independent errors. If each tree has error rate < 0.5 and errors are independent, the ensemble error decreases as B increases.

7.2 Random Forest

Random Forest = Bagging + **Random Feature Subsampling at each split**.

An additional randomisation step is applied: at each split in each tree, instead of evaluating *all* n features, the algorithm evaluates only a **random subset of k features** where:

$$\boxed{k = \lfloor \sqrt{n} \rfloor}$$

(Common default; for regression, often $k = \lfloor n/3 \rfloor$.)

Full Random Forest Algorithm:

1. Given training set of size m and n features.
2. For each tree $b = 1, \ldots, B$:
   (a) Draw a bootstrap sample S_b of m examples.
   (b) Build a decision tree on S_b:
      - At each node, randomly select $k = \lfloor \sqrt{n} \rfloor$ features.
      - Split on the best among *only those k features.*
3. Aggregate predictions from all B trees:
   - Classification: majority vote.
   - Regression: mean.

Alternative / Comparison

Why randomly subsample features?
Even with bootstrapping, if a very strong feature exists, most trees will split on it at the root, making the trees highly correlated. Correlated errors don't cancel out. By forcing each split to consider only a random subset of features, the trees are *decorrelated*: even if one feature is dominant, individual trees will use different features and thus make different errors. **Decorrelated trees $\Rightarrow$ larger variance reduction.**

	Bagging	Random Forest
Training data	Bootstrap samples	Bootstrap samples
Feature selection	All n features per split	$k = \sqrt{n}$ random features
Tree correlation	Higher	Lower
Variance reduction	Good	Better

Example

Intuition with a vote analogy: Imagine $B = 100$ friends each study a slightly different summary of the same course material. You ask them all to predict whether a student will pass.

- If all friends read the same summary (no diversity), they all make the same errors — the majority vote still has those errors.
- If each friend read a different portion (diversity), different errors cancel out — the majority vote is much more accurate.

Random Forest maximises diversity by *both* varying the training set (bootstrap) *and* varying the features each tree uses.

8 Boosting and XGBoost

8.1 The Boosting Concept

While bagging trains trees *independently in parallel*, **boosting** trains trees *sequentially*: each new tree focuses on correcting the mistakes of the previous ones.

Core idea:

1. Train the first tree.
2. Identify examples the first tree misclassifies.
3. **Increase the weight** of those misclassified examples (make them "harder" for the next tree to ignore).
4. Train the second tree on the reweighted data — it now focuses on the previous errors.
5. Repeat for B rounds.

6. Combine all B trees using a **weighted sum**: later trees (or more accurate trees) get higher weights.

Key Idea

Bagging vs. Boosting:

- **Bagging**: Trees are independent; errors reduced by *averaging*. Primary goal: reduce *variance*.
- **Boosting**: Trees are sequential; errors corrected *in sequence*. Primary goal: reduce *bias* (and also variance).

8.2 XGBoost (Extreme Gradient Boosting)

XGBoost is a highly optimised, production-grade implementation of gradient boosting that has become the dominant algorithm in machine learning competitions and industry.

Key features:

- **Fast:** Uses a parallel tree-building algorithm and hardware-level optimisations.
- **Regularisation:** Adds L1 and L2 penalties on tree weights to prevent overfitting.
- **Automatic split selection:** Uses an approximate greedy algorithm to efficiently find the best splits.
- **Automatic stopping:** Monitors validation error and stops when improvement plateaus.
- **Missing value handling:** Automatically learns the best direction for missing-value examples.

8.3 XGBoost Algorithm — Step by Step

1. **Initialise:** Train the first decision tree on the original dataset.
2. **Identify mistakes:** Compute the *residuals* (errors) of the current model's predictions. In classification, assign **higher weights** to misclassified examples.
3. **Train corrector tree:** Fit a new tree to predict the residuals (or to focus on weighted errors).
4. **Update model:** Add the new tree's predictions to the ensemble (scaled by a *learning rate* $\eta \in (0, 1]$).
5. **Repeat** steps 2–4 for B iterations.
6. **Final prediction** = weighted sum of all B trees:

$$\hat{y} = \sum_{b=1}^{B} \eta \cdot f_b(\mathbf{x})$$

where $f_b(\mathbf{x})$ is the prediction of tree b.

Key Idea

Learning rate η (also called shrinkage): Each new tree's contribution is scaled down by η. A small η (e.g., 0.01–0.1) means each tree makes a small step — learning is slow but more robust. A large η (e.g., 0.5–1.0) takes larger steps but may overshoot and overfit. Typical choice: $\eta = 0.1$ with $B = 100$–500 trees.

8.4 XGBoost Worked Example

Example

Dataset (modified — note D is now Pass/1):

Student	Study Hours	Attendance (%)	Pass (1) / Fail (0)
A	1	50	0 (Fail)
B	5	80	1 (Pass)
C	6	85	1 (Pass)
D	2	40	**1 (Pass)**
E	3	60	0 (Fail)
F	8	90	1 (Pass)

Overall: 4 Pass, 2 Fail.

XGBoost Iteration 1 — Initial tree:

Split on Study Hours = 3.5:

- Left (≤ 3.5): A(Fail), D(Pass), E(Fail) $\rightarrow$ majority = Fail $\rightarrow$ predict **Fail**.
- Right (> 3.5): B(Pass), C(Pass), F(Pass) $\rightarrow$ majority = Pass $\rightarrow$ predict **Pass** (pure leaf).

Errors after Tree 1:

- A: predicted Fail, actual Fail $\rightarrow$ correct.
- D: predicted Fail, actual **Pass** $\rightarrow$ **misclassified**.
- E: predicted Fail, actual Fail $\rightarrow$ correct.
- B, C, F: all correct.

Only student D is misclassified. **XGBoost assigns D a higher weight for the next iteration.**

XGBoost Iteration 2 — Corrector tree:

The second tree is trained on the same dataset but with D now weighted more heavily. It is likely to isolate D: split Study Hours ≤ 2.5 (separating D from A, E):

- Node ≤ 2.5: A(Fail, wt=1), D(Pass, wt=high) $\rightarrow$ D's higher weight forces predict **Pass**.
- Node > 2.5 in [≤ 3.5]: E(Fail) $\rightarrow$ predict **Fail**.

The ensemble (Tree 1 + $\eta\cdot$ Tree 2) now correctly classifies D while maintaining accuracy on others.

Final model: weighted combination of both trees. Each subsequent tree further reduces residual errors.

9 Comparison: Bagging vs. Random Forest vs. Boosting

Property	Bagging	Random Forest	Boosting (XGBoost)
Training order	Parallel	Parallel	Sequential
Training data	Bootstrap	Bootstrap	Weighted/Reweighted
Feature subset	All	$k = \sqrt{n}$	All (by default)
Primary goal	Reduce variance	Reduce variance	Reduce bias
Combination	Majority/Average	Majority/Average	Weighted sum
Regularisation	None intrinsic	None intrinsic	L1/L2 built in
Overfitting risk	Moderate	Low	Low (with η)
Speed	Fast	Fast	Slower (sequential)
Interpreting output	Harder than tree	Harder than tree	Hardest
Accuracy	Good	Very good	Often best

Key Idea

Practical rule of thumb:

- Start with **Random Forest** — robust, fast, few hyperparameters, hard to make it overfit, works well out-of-the-box.
- Switch to **XGBoost** when you need maximum accuracy and are willing to tune hyperparameters (η, B, max depth, regularisation).

10 Practice Questions

Practice Questions

Topic: Pruning

Q1 (Easy): What is the difference between pre-pruning and post-pruning? Give one advantage of each.
Answer: **Pre-pruning** stops tree growth early (before it overcomplicates) using stopping rules like max depth. *Advantage*: Fast; does not build a large tree.
Post-pruning builds the full tree then removes unnecessary branches. *Advantage*: More accurate pruning decisions because the full structure is visible before cutting.

Q2 (Medium): A split creates 2 subsets: Subset 1 has 6 Pass, 2 Fail; Subset 2 has 4 Pass, 8 Fail. The parent node has 10 Pass, 10 Fail ($p = 10, n = 10$). Compute the expected counts for Subset 1 ($p_k = 6, n_k = 2$, total $= 8$).
Answer: $\hat{p}_1 = p \times (p_1 + n_1)/(p + n) = 10 \times 8/20 = 4.0$
$\hat{n}_1 = n \times (p_1 + n_1)/(p + n) = 10 \times 8/20 = 4.0$

(Contribution to χ^2 from Subset 1: $(6-4)^2/4 + (2-4)^2/4 = 1.0 + 1.0 = 2.0$.)

Q3 (Exam-level): Using the data in Q2, compute the full χ^2 statistic and determine whether the split is significant at $\alpha = 0.05$.
Answer: Subset 2: $p_2 = 4, n_2 = 8$, total $= 12$. $\hat{p}_2 = 10 \times 12/20 = 6.0$; $\hat{n}_2 = 10 \times 12/20 = 6.0$.
$\chi^2 = \underbrace{2.0}_{\text{Subset 1}} + \frac{(4-6)^2}{6} + \frac{(8-6)^2}{6} = 2.0 + 0.667 + 0.667 = \mathbf{3.334}$
DF $= 2 - 1 = 1$. Critical value at $\alpha = 0.05$, DF=1: 3.841.
Since $3.334 < 3.841$: fail to reject H_0. The attribute is **not significant at 5%**; prune the split.

Topic: Random Forests

Q4 (Easy): A dataset has $n = 16$ features. How many features does a Random Forest evaluate at each split?
Answer: $k = \lfloor\sqrt{16}\rfloor = 4$ features.

Q5 (Medium): Explain why decorrelating decision trees in a Random Forest leads to better predictions than simply using multiple identical trees trained on the full dataset.
Answer: Identical trees make the same predictions; majority vote does not improve on a single tree. Diversity in predictions is required for errors to cancel. One tree might always split first on "Study Hours" and misclassify a particular type of student. Decorrelated trees sometimes do not see "Study Hours" and find other patterns — their errors are independent. Independent errors cancel under majority voting; correlated errors do not.

Q6 (Exam-level): A training set has $m = 100$ examples with $n = 9$ features. You build a Random Forest with $B = 50$ trees. Describe exactly what happens during construction.
Answer: For each of 50 trees:

1. Draw 100 examples *with replacement* from the training set (some appear multiple times, some don't appear).
2. Build a decision tree on this sample. At each node, randomly pick $k = \lfloor\sqrt{9}\rfloor = 3$ features and split on the best among those 3.
3. Grow the tree to full depth (or until a stopping criterion is met).

Each of the 50 trees is independent and trained on a slightly different dataset with a random feature subset at each split.
To predict: pass a new example through all 50 trees, collect 50 class votes, return the majority.

Topic: Boosting and XGBoost

Q7 (Medium): What is the role of the learning rate η in XGBoost? What happens if η is too large or too small?
Answer: η scales each new tree's contribution to the ensemble.

- Too large ($\eta \to 1$): each tree has full weight; the model converges fast but can overshoot and overfit — it relies too heavily on each individual tree.
- Too small ($\eta \to 0$): each tree contributes very little; many more trees are needed (B must be large) — slow training but smoother, better generalisation.
- Typical choice: $\eta = 0.1$ with $B = 100$–500.

Q8 (Exam-level): Explain in your own words how boosting reduces *bias*, while bagging reduces *variance*. Why does a single deep decision tree exhibit high variance but not necessarily high bias?
Answer: **Bias** = systematic error from wrong assumptions (underfitting); **variance** = sensitivity to training data changes (overfitting).
A single deep tree has **low bias** (it can memorise any training set) but **high variance** (different training sets produce very different trees).
Bagging trains many independent trees on different data subsets and averages predictions. Each tree still fits its data closely (low bias), but their prediction variance cancels under averaging — giving low variance overall. Bagging does not reduce bias because each individual tree is still complex.
Boosting uses shallow trees (low variance individually) and sequentially corrects remaining errors. Initially the ensemble underfits (high bias); each round corrects systematic errors, *reducing bias* while the shallow trees limit overfitting.

11 Key Takeaways — Week 7

Quick Summary

- **C4.5 missing values:** $\text{Gain}(A) = (N_{\text{known}}/N) \times \text{Gain}_{\text{known}}$. Missing examples are sent fractionally to all branches; prediction is a weighted branch probability.
- **CART Regression:** split to minimise weighted MSE; leaf prediction = mean of labels in leaf. Best split often Study Hours ≤ 3.5, reducing MSE from 453.8 to 60.4 on the student dataset.
- **Pruning prevents overfitting.** Pre-pruning stops early; post-pruning removes branches after full tree construction.
- **χ^2 pre-pruning test:** Compare actual vs. expected class counts under the null (attribute is irrelevant). Prune if $\chi^2 < \chi^2_{\text{crit}}$ (attribute insignificant); keep otherwise. DF $= d - 1$ where d = number of subsets.
- **C4.5 post-pruning:** Grow full tree; compute ObservedError $= E/N$; add CI adjustment to get Estimated Error; prune if Error(leaf) $\leq$ Error(subtree).
- **Decision trees have high variance** — small data changes can flip the entire tree structure.
- **Ensemble methods** combine many trees to reduce variance (bagging/Random Forest) or bias (boosting).

- **Bagging:** Bootstrap sample m with replacement; train B independent trees; majority vote / average.
- **Random Forest** = Bagging + random feature subset ($k = \lfloor\sqrt{n}\rfloor$) per split. Feature subsampling *decorrelates* trees, amplifying variance reduction.
- **Boosting:** Sequential trees; each tree corrects previous errors by upweighting misclassified examples. Reduces bias.
- **XGBoost:** Advanced gradient boosting with L1/L2 regularisation, learning rate η, and fast parallel building. Final prediction $= \sum_b \eta \cdot f_b(\mathbf{x})$.
- **In practice:** Random Forest as a first choice; XGBoost when maximum accuracy is needed.